

Task 1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
vedadripinninti@MacbookAir: ~/s + v
vedadripinninti@MacbookAir:~$ mkdir skill_week7
vedadripinninti@MacbookAir:~$ cd skill_week7
vedadripinninti@MacbookAir:~/skill_week7$ nano 1.c
```

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Usage: %s input.txt\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1], O_RDONLY);

    if (fd < 0) {
        perror("File not found");
        return 1;
    }

    int old_stdin = dup(STDIN_FILENO);

    dup2(fd, STDIN_FILENO);
    close(fd);

    char str[100];

    printf("Data from file:\n");
    while (fgets(str, sizeof(str), stdin))
        printf("%s", str);

    /* Restore stdin */
    dup2(old_stdin, STDIN_FILENO);
    close(old_stdin);

    return 0;
}
```

COMPILE AND RUN:

```
vedadripinninti@MacbookAir: ~/s + v
vedadripinninti@MacbookAir:~$ mkdir skill_week7
vedadripinninti@MacbookAir:~$ cd skill_week7
vedadripinninti@MacbookAir:~/skill_week7$ nano 1.c
vedadripinninti@MacbookAir:~/skill_week7$ gcc 1.c -o 1
vedadripinninti@MacbookAir:~/skill_week7$ ./1
```

OUTPUT:

```
vedadripinninti@MacbookAir: ~/s + v
vedadripinninti@MacbookAir:~$ mkdir skill_week7
vedadripinninti@MacbookAir:~$ cd skill_week7
vedadripinninti@MacbookAir:~/skill_week7$ nano 1.c
vedadripinninti@MacbookAir:~/skill_week7$ gcc 1.c -o 1
vedadripinninti@MacbookAir:~/skill_week7$ ./1
Usage: ./1 input.txt
vedadripinninti@MacbookAir:~/skill_week7$
```

```
vedadripinninti@MacbookAir:~/skill_week7$
vedadripinninti@MacbookAir:~/skill_week7$ echo "Hello from input file" > input.txt
vedadripinninti@MacbookAir:~/skill_week7$ ./1 input.txt
Data from file:
Hello from input file
```

```
vedadripinninti@MacbookAir:~/skill_week7$
```

Task 2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd, old_stdout;
    char *file = "output.txt";

    // Create output file
    fd = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Save original stdout
    old_stdout = dup(STDOUT_FILENO);

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);
    close(fd);

    // This output goes to output.txt
    printf("Hello from C program\n");
    printf("Output redirection successful\n");

    fflush(stdout);

    // Restore stdout
    dup2(old_stdout, STDOUT_FILENO);
    close(old_stdout);

    printf("Output restored to terminal\n");

    return 0;
}
```

OUTPUT:

```
vedadripinninti@MacbookAir:~/skill_week7$
vedadripinninti@MacbookAir:~/skill_week7$ nano 2.c
vedadripinninti@MacbookAir:~/skill_week7$ gcc 2.c -o 2
vedadripinninti@MacbookAir:~/skill_week7$ ./2
Output restored to terminal
vedadripinninti@MacbookAir:~/skill_week7$
```

```
vedadripinninti@MacbookAir:~/skill_week7$
vedadripinninti@MacbookAir:~/skill_week7$ cat output.txt
Hello from C program
Output redirection successful
vedadripinninti@MacbookAir:~/skill_week7$
```

SESSION-14

Task 1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Usage: %s file.txt\n", argv[0]);
    }
}
```

```

        return 1;
    }

    // Open file in append mode
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Save stdout
    int old_stdout = dup(STDOUT_FILENO);

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);
    close(fd);

    // Append data
    printf("New line appended to file.\n");
    fflush(stdout);

    // Restore stdout
    dup2(old_stdout, STDOUT_FILENO);
    close(old_stdout);

    printf("Append completed.\n");

    return 0;
}

```

OUTPUT:

```

vedadripinninti@MacbookAir:~/skill_week7$ nano 3.c
vedadripinninti@MacbookAir:~/skill_week7$ gcc 3.c -o 3
vedadripinninti@MacbookAir:~/skill_week7$ ./3
Usage: ./3 file.txt
vedadripinninti@MacbookAir:~/skill_week7$

```

```

vedadripinninti@MacbookAir:~/skill_week7$
vedadripinninti@MacbookAir:~/skill_week7$ ./3 output.txt
Append completed.
vedadripinninti@MacbookAir:~/skill_week7$ cat output.txt
Hello from C program
Output redirection successful
New line appended to file.
vedadripinninti@MacbookAir:~/skill_week7$

```

Task 2: TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

CODE:

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd = open("error.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Cannot create error file");
        return 1;
    }

    // Save original stderr
    int old_stderr = dup(STDERR_FILENO);

    // Redirect stderr to file
    dup2(fd, STDERR_FILENO);
    close(fd);

    // Generate an error
    FILE *fp = fopen("missing.txt", "r");

```

```
if (fp == NULL)
    perror("Error opening file");

fflush(stderr);

// Restore stderr
dup2(old_stderr, STDERR_FILENO);
close(old_stderr);

printf("Error redirection completed.\n");

return 0;
}
```

OUTPUT:

```
vedadripinninti@MacbookAir:~/skill_week7$ nano 4.c
vedadripinninti@MacbookAir:~/skill_week7$ gcc 4.c -o 4
vedadripinninti@MacbookAir:~/skill_week7$ ./4
Error redirection completed.
vedadripinninti@MacbookAir:~/skill_week7$
```